

Linux Server Operations Lab

Phase 9 レビュー

障害発生 → 検知 → Excel から復旧 → 再確認

Phase 9 の目的

意図的に障害を起こし、これまで作成した Excel・Python・Ansible・Linux の仕組みを使って、障害検知から復旧、再点検までを一連の運用フローとして確認する。

1. 今回実施したこと

対象は SV001 / tokyo の nginx。まず nginx を意図的に停止し、点検スクリプトで異常を検知。その後、Excel の操作指示から nginx を再起動し、最後に再点検して正常復帰を確認した。

段階	内容	結果
障害発生	tokyo の nginx を停止	inactive
障害検知	write_results.py を実行	tokyo のみ FAILED
復旧指示	REQ004 / RESTART_NGINX / YES	SUCCESS
再確認	write_results.py を再実行	3 台すべて OK

2. 障害を意図的に発生させる

tokyo サーバへ SSH 接続し、nginx を停止した。検証環境だからこそ、実際に障害を作って挙動を確認できる。

```
ssh tokyo@192.168.56.11
sudo systemctl stop nginx
systemctl is-active nginx
```

確認結果は inactive。これで tokyo の nginx が停止した状態を作ることができた。

3. Ansible で障害を直接確認

まず 3 台への SSH 疎通を確認した。

```
ansible servers \
-i ~/linux-server-operations-lab/inventory/hosts_generated.ini \
-m ping \
-T 5
```

omiya / yokohama / tokyo の 3 台すべてが SUCCESS / pong。つまり、今回の異常は「サーバへ接続できない」障害ではなく、「nginx サービスだけが停止している」障害だと切り分けられた。

次に nginx の状態を 3 台まとめて確認した。

```
ansible servers \
-i ~/linux-server-operations-lab/inventory/hosts_generated.ini \
-a "systemctl is-active nginx" \
-T 5
```

結果は、yokohama と omiya が active、tokyo が FAILED / rc=3 / inactive。障害対象が tokyo の nginx に限定されていることを確認できた。

4. write_results.py の待ち時間問題

最初に write_results.py を実行した際、処理が長時間待ち状態となり、Ctrl+C で停止した。Python 側では KeyboardInterrupt が発生し、subprocess.run() 内で Ansible の終了を待っていたことが分かった。

```
disk_result = subprocess.run(
    [
        "ansible", "servers", "-i", inventory,
        "-a", "df -P /", "-T", "5"
    ],
    capture_output=True,
    text=True,
    timeout=30
)

nginx_result = subprocess.run(
    [
        "ansible", "servers", "-i", inventory,
        "-a", "systemctl is-active nginx", "-T", "5"
    ],
    capture_output=True,
    text=True,
    timeout=30
)
```

5. タイムアウトを二段階で設定

Ansible 側の -T 5 は SSH 接続待ちを最大 5 秒に制限する。Python 側の timeout=30 は subprocess.run() 全体が 30 秒以上待ち続けないようにする。

これにより、対象サーバやネットワークに問題があっても、運用スクリプトが無期限に停止するリスクを下げられる。

6. Excel へ障害結果を書き戻す

修正後に write_results.py を再実行した。

```
SV001 tokyo Disk=25% nginx=FAILED Result=FAILED
SV002 yokohama Disk=17% nginx=OK Result=OK
SV003 omiya Disk=16% nginx=OK Result=OK

Excel write completed
```

tokyo だけが FAILED となり、yokohama / omiya は OK のまま。Linux 上の障害を Ansible で取得し、Python で判定して、Excel の 02_CHECK_RESULTS に反映する流れを確認できた。

7. Excel から復旧指示を出す

RequestID	ServerID	Action	Execute
REQ004	SV001	RESTART_NGINX	YES

execute_operations.py を実行すると、REQ004 だけが処理され、SUCCESS となった。

```
Executing REQ004: SV001 -> tokyo -> RESTART_NGINX
SUCCESS

Operation processing completed
```

既存の REQ001～REQ003 は SUCCESS 済みだったため、再実行防止ロジックによってスキップされた。

8. 復旧後の再点検

```
SV001 tokyo Disk=25% nginx=OK Result=OK  
SV002 yokohama Disk=16% nginx=OK Result=OK  
SV003 omiya Disk=16% nginx=OK Result=OK
```

Excel write completed

tokyo の nginx が OK に戻り、3 台すべて正常状態へ復帰した。

9. Phase 9 で完成した障害対応フロー

```
障害発生  
↓  
Ansible で状態取得  
↓  
Python で OK / FAILED 判定  
↓  
Excel へ障害表示  
↓  
Excel で復旧指示  
↓  
Python が指示を読み込む  
↓  
Ansible で Linux を復旧  
↓  
再点検  
↓  
Excel が OK へ戻る
```

10. トラブルシュートで学んだこと

- nginx が停止していても SSH 疎通自体は正常なため、接続障害かサービス障害かを切り分ける必要がある。
- Ansible の FAILED は必ずしも「Ansible 自体の異常」ではない。今回の rc=3 / inactive は systemctl is-active がサービス停止を正しく返した結果。
- 自動化スクリプトには、対象システムだけでなく「自動化処理自身が止まらない設計」も必要。
- タイムアウトは障害対応時の無限待ちを防ぐ重要な防御になる。

11. 今回の主な実行コマンド

```
sudo systemctl stop nginx
systemctl is-active nginx

ansible servers \
-i ~/linux-server-operations-lab/inventory/hosts_generated.ini \
-m ping -T 5

ansible servers \
-i ~/linux-server-operations-lab/inventory/hosts_generated.ini \
-a "systemctl is-active nginx" -T 5

python3 ~/linux-server-operations-lab/scripts/write_results.py
python3 ~/linux-server-operations-lab/scripts/execute_operations.py
```

12. 実務での応用

今回の仕組みは nginx だけでなく、SSH、Web アプリ、バッチ、ディスク使用率、バックアップジョブなどにも応用できる。Excel を運用窓口として使う場合でも、実際の処理は Python と Ansible に任せることで、操作と自動化を分離できる。

また、復旧 Action を限定しておけば、誰でも自由な Linux コマンドを実行できる設計より安全性を高められる。

13. Phase 9 で学んだこと

- 障害対応は「検知 → 切り分け → 復旧 → 再確認」までが1セット。
- 正常系だけでなく、意図的に障害を起こすことで自動化の弱点を発見できる。
- Excel は単なる結果表示だけでなく、復旧指示の入口としても利用できる。
- SUCCESS 済みの依頼を再実行しない設計により、不要なサービス再起動を防げる。

14. 次の Phase

Phase 10：Excel ダッシュボード化。01_SERVER_MASTER、02_CHECK_RESULTS、03_OPERATION_REQUESTS をまとめ、サーバ状態・最新点検時刻・操作履歴などを見やすく確認できる最終画面を作成する。

15. note 記事候補

- Excel から Linux 障害を復旧する仕組みを作ってみた
- Ansible で nginx 停止を検知し、Python で Excel へ FAILED を書き戻す
- 自動化スクリプトにタイムアウトを入れる理由
- Excel × Python × Ansible で作る簡易サーバ運用コンソール